

Some IDEs will have refactoring features that can identify names based on how your program uses them. For example, a common feature is a “rename” tool that can differentiate between variables named `num` and `number`, as well as between local `num` and global `num` variables. But you should still choose names as though these tools weren’t available.

Keeping this rule in mind will naturally help you pick descriptive names instead of generic ones. The name `email` is vague, so consider a more descriptive name like `emailAddress`, `downloadEmailAttachment`, `emailMessage`, OR `replyToAddress`. Not only would such a name be more precise, it would be easier to find in your source code files as well.

Avoid Jokes, Puns, and Cultural References

At one of my previous software jobs, our codebase contained a function named `gooseDownload()`. I had no idea what this meant, because the product we were creating had nothing to do with birds or the downloading of birds. When I found the more-senior co-worker who had originally written the function, he explained that *goose* was meant as a verb, as in “goose the engine.” I had no idea what this phrase meant, either. He had to further explain that “goose the engine” was automotive jargon that meant press down on the gas pedal to make the engine go faster. Thus, `gooseDownload()` was a function to make downloads go faster. I nodded my head and went back to my desk. Years later, after this co-worker left the company, I renamed his function to `increaseDownloadSpeed()`.

When choosing names in your program, you might be tempted to use jokes, puns, or cultural references to add some levity to your code. Don’t do this. Jokes can be hard to convey in text, and the joke probably won’t be as funny in the future. Puns can also be easy to miss, and handling repeat bug reports from co-workers who confused a pun for a typo can be quite punishing.

Culture-specific references can get in the way of communicating your code’s intent clearly. The internet makes it easier than ever to share source code with strangers around the world who won’t necessarily be fluent in English or understand English jokes. As noted earlier in the chapter, the names `spam`, `eggs`, and `bacon` used in Python documentation reference a Monty